



Replication

*<Practice provides hands-on **experience** with PostgreSQL Replication >*



About me:

- Java Developer
- Co-Organizer of Codeus community

What is WAL?

Write-Ahead Log (WAL) is a logging technique where all changes are first written to a log before being applied to the data files.

Core principle

Change → WAL Record → Flush to Disk → Apply to Data

Purpose of WAL

ACID
Properties

Crash
Recovery

Data
Replication

Point-in-Time
Recovery

Ensures data consistency and durability

Allows database recovery after system failures

Foundation for both physical and logical replication

Enables restoration to specific moments

Benefits of WAL

Guarantees data durability

Enables crash recovery

Supports replication mechanisms

Provides audit trail of all changes

WAL Logging Levels

`wal_level = minimal`

Purpose: Minimal info for crash recovery only

Limitation: Replication NOT supported

Use Case: Single-instance DB without replication needs

WAL Logging Levels

`wal_level = replica`

Purpose: Sufficient for physical replication

Features: Supports Hot Standby servers

Use Case: Physical replication scenarios

WAL Logging Levels

`wal_level = logical`

Purpose: Supports logical replication

Features: Has additional info for logical decoding

Use Case: Logical and selective data replication

What is LSN?

LSN (Log Sequence Number) - is a unique identifier for the position of a record in WAL files

Structure

```
LSN = 0/1A2B3C4D
      |  |
      |  └─ Offset in file (hex)
      └─ WAL file number (hex)
```

Key Functions

```
-- Current WAL position
SELECT pg_current_wal_lsn();

-- Last checkpoint position
SELECT checkpoint_lsn FROM pg_control_checkpoint();

-- Calculate LSN difference (lag in bytes)
SELECT pg_wal_lsn_diff('0/2000000', '0/1000000');

-- Convert LSN to filename
SELECT pg_walfile_name('0/1A2B3C4D');
```

Usage LSN in Replication

Track replication progress

Identify WAL position

Calculate replication lag

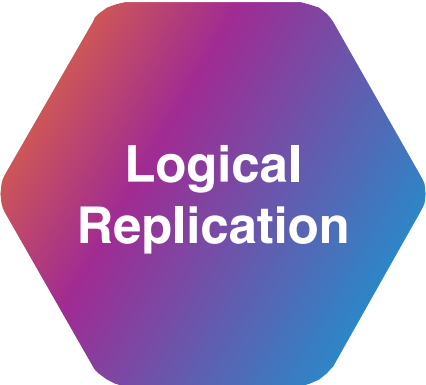
Ensure consistency

Replication Types



**Streaming
Replication**

Physical - Byte-level WAL transmission



**Logical
Replication**

Operation-level data synchronisation

Streaming Replication (Physical)

Streaming Replication transmits WAL records from Primary to Standby servers in real-time over TCP connections.

How it works?

Primary Server:

```
WAL: [A1B2C3D4]
Page 123, 0ff 45
```

—streaming—→

```
WAL: [A1B2C3D4]
Page 123, 0ff 45
```

Standby Server

Key Characteristics

Byte-level copying of WAL records

Real-time transmission via network streaming

Hot Standby capability (read-only queries on standby)

Exact replica of the primary server

Requirements

Identical PostgreSQL version on all servers

Same architecture (x86, ARM, etc.)

Identical database schema and structure

Same extensions and configurations

Streaming Replication Use Cases

High Availability (failover scenarios)

Load Balancing (read queries on standby)

Disaster Recovery (exact backup)

Geographic distribution (identical replicas)

Logical Replication

Logical Replication decodes WAL records into logical operations (INSERT, UPDATE, DELETE) and transmits these operations to subscribers.

How it works?

Publisher:

```
WAL: [A1B2C3D4]
Page 123, Off 45
```

↓
▼ (WAL Decoding)

```
INSERT INTO
users VALUES
(1, 'John'...)
```

—logical—→

```
INSERT INTO
users VALUES
(1, 'John'...)
```

Subscriber

What Gets Replicated in Logical Replication

Selected tables only (via Publications)

DML operations (INSERT, UPDATE, DELETE)

Data changes in chosen databases

Configurable operation types (insert-only, etc.)



What Does NOT Get Replicated in Logical Replication

DDL operations (schema changes)

Sequences (SERIAL values)

Functions and triggers

User roles and permissions



Logical Replication Use Cases

Selective data synchronisation

Cross-version upgrades (PostgreSQL 13 → 16)

Data warehousing (specific tables only)

Cross-platform replication

Data transformation pipelines

Performance Considerations

Streaming Replication

- ✓ Lower overhead (binary copying)
- ✓ Faster replication (no decoding required)
- ✓ Network efficient (compressed WAL transmission)
- ⚠ Limited by WAL generation rate

Logical Replication

- ⚠ Higher CPU usage (WAL decoding process)
- ⚠ More network traffic (SQL statements transmission)
- ✓ Flexible filtering reduces unnecessary data transfer
- ⚠ Can introduce replication lag on high-volume systems

Best Practices & Recommendations

Configuration Best Practices

Set appropriate wal_level based on replication needs

Configure sufficient wal_buffers (typically 16MB)

Enable WAL archiving for production systems

Monitor WAL generation rate and storage

Tune checkpoint parameters for your workload

Monitoring Recommendations

Track WAL file count and size

Monitor replication lag regularly

Check archive command success rate

Alert on WAL disk space usage

Monitor checkpoint frequency and duration

Streaming Replication Best Practices

Monitor replication lag regularly

Configure sufficient WAL retention

Use replication slots for guaranteed delivery

Plan failover procedures in advance

Test recovery scenarios periodically

Logical Replication Best Practices

Design Publications carefully(avoid unnecessary tables)

Monitor replication slot growth

Handle schema changes manually

Plan for conflict resolution

Test subscriber performance under load

Thank you

- Author: Yevhenii Fortuna
- My LinkedIn: <https://www.linkedin.com/in/yevhenii-fortuna/>
- June 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)